



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

ADVANCED JAVASCRIPT BACKEND FRAMEWORKS (BTOE561T5P)

CIA 3 - Project Report

“Book My Bite” - The Royal Bistro

Restaurant Table Reservation & Food Ordering System

5th Semester

Submitted By:

Sr No.	Name	Register Number	Department	Class
1	Alicia Theresa Pereira	2462028	Artificial Intelligence and Data Science Engineering	5 BTCS AIML C
2	Alan Baiju K	2462022		
3	Albert John	2462025		
4	Ahoan Sarkar			

Christ (Deemed to be University), Bangalore

<https://github.com/aliciaa-20/book-my-bite>

Restaurant Table Reservation & Food Ordering System

Github Link:

<https://github.com/aliciaa-20/book-my-bite>

1. Abstract

Book My Bite (branded in-application as “The Royal Bistro”) is a full-stack, role-restricted, multi-branch restaurant management platform built on the MERN-adjacent stack of Node.js, Express.js, and MongoDB with Mongoose as the object data modelling layer. The system is designed to digitise the end-to-end lifecycle of a modern restaurant's front-of-house and back-of-house operations, spanning customer-facing menu browsing and table reservation, dine-in and takeaway order placement, a real-time Kitchen Display System (KDS) for order fulfilment, itemised billing with tax and service-charge computation, post-meal feedback collection, and manager-facing analytics built on MongoDB's aggregation framework.

The backend exposes a versioned REST API secured with JSON Web Token (JWT) authentication, bcrypt password hashing, and role-based access control (RBAC) across four personas — customer, kitchen staff, manager, and administrator. The project was implemented as the Continuous Internal Assessment 3 (CIA 3) deliverable for the Advanced JavaScript Backend Frameworks (BTOE561T5P) course, and demonstrates applied use of Express middleware chains, schema validation, centralised error handling, automated testing with Jest and Supertest, and API documentation via an exported Postman collection.

2. Objectives

The project was undertaken with the following objectives:

- Design and implement a RESTful backend for restaurant operations using Node.js and Express.js, following modular MVC-style organisation (routes, controllers, models, middleware).
- Model a multi-branch restaurant domain in MongoDB using Mongoose, applying deliberate embedding vs. referencing decisions to balance data integrity, historical accuracy, and query performance.
- Implement secure authentication and authorisation using JWT and bcrypt, with granular role-based permissions for customer, kitchen, manager, and admin users.
- Automate table reservation conflict detection to eliminate double-booking, and enforce a strict order-status state machine to reflect real kitchen workflows.
- Provide real-time operational visibility to kitchen staff through a FIFO Kitchen Display System queue, and to managers through aggregation-driven sales and traffic analytics.
- Validate the API surface through automated Jest/Supertest test suites and a shareable Postman collection, and demonstrate the system through a lightweight web front end.

3. Problem Statement

In the modern hospitality industry, restaurants that rely on manual, paper-based, or fragmented digital processes routinely encounter table double-booking conflicts, high table turnaround latency, miscommunication between the dining room and the kitchen, inconsistent bill calculation, and an absence of real-time insight into dish-level profitability and peak traffic periods.

Book My Bite addresses this gap by providing a single, scalable backend platform — The Royal Bistro — that lets customers browse a live, categorised menu, reserve dining tables with automated

conflict detection, and place dine-in or takeaway orders. The system orchestrates a real-time Kitchen Display System queue for kitchen staff, enforces strict order-status transitions (Placed → Preparing → Ready → Served), computes itemised bills including GST and service charges, and gives managers executive-level MongoDB aggregation analytics on daily revenue, dish popularity, and peak dining hours.

4. Scope of the Project

4.1 In Scope

- User registration, authentication, and profile management with role-based access control.
- Multi-branch restaurant, table, and menu-item management with CRUD operations.
- Table reservation with slot-conflict detection, cancellation, and rescheduling.
- Dine-in and takeaway order placement, order-status workflow, and kitchen order queue.
- Billing computation (subtotal, discounts, GST, service charge) and mock payment tracking.
- Customer order history and post-meal feedback / ratings.
- Manager and admin analytics dashboards built on MongoDB aggregation pipelines.
- Automated backend testing (Jest, Supertest) and a Postman collection for API verification.

5. Target Audience

- Restaurant customers who want to browse menus, reserve tables, and place dine-in or takeaway orders.
- Kitchen staff who need a real-time, prioritised queue of incoming orders to prepare.
- Branch managers responsible for menu, table, and staff oversight and daily sales performance.
- System administrators managing multiple restaurant branches, user roles, and platform-wide configuration.

6. Tools & Technologies Used

Category	Technology
Runtime & Framework	Node.js (v18+), Express.js (v4.x)
Database & ODM	MongoDB with Mongoose ODM (v8.x)
Database Resilience	mongodb-memory-server auto-fallback for zero-config demo/tests; standard MongoDB / Atlas support
Authentication & Security	JSON Web Tokens (JWT), bcrypt.js password hashing, Helmet security headers, CORS
Validation	Express-Validator schema validation middleware
Frontend / Demo UI	Vanilla HTML5, CSS3 (glassmorphism), modern JavaScript (ES6+), Font Awesome, Chart.js
Testing & Tooling	Jest, Supertest, Postman Collection v2.1, Morgan HTTP logger
Version Control	Git and GitHub

7. Key Features

7.1 User Authentication & Role Management

Registration and login are secured with bcrypt password hashing and JWT-based session tokens. Four roles — customer, kitchen, manager, and admin — are enforced across every protected route via role-checking middleware, and users can view and update their own profiles.

7.2 Menu Management

Managers and admins can create, update, and remove menu items with category tags, vegetarian/non-vegetarian flags, price, preparation time, and live availability toggling. Customers browse the menu with category and search filters.

7.3 Table Management

Tables are modelled per branch with seating capacity (2–10), zone grouping (Indoor, Patio, Rooftop, VIP), and active/inactive status, allowing managers to maintain an accurate seating inventory.

7.4 Table Reservation

Customers check available slots by date and guest count; the reservation engine performs conflict detection against existing Confirmed or Seated bookings for the same branch, date, and time slot, and auto-allocates a best-fit table by party size, rejecting collisions with an HTTP 409 response.

7.5 Food Ordering

Customers place dine-in orders (linked to a table) or takeaway orders, with per-item preparation instructions. Menu item name, price, and instructions are captured as an immutable snapshot at order time so that later menu changes never alter historical receipts.

7.6 Order Tracking

Every order carries a status field and an embedded status-history audit trail, letting customers and staff track progress through the fulfilment lifecycle in real time.

7.7 Kitchen Display System

Kitchen staff work from a live FIFO queue of incoming tickets with preparation countdowns and one-click status advancement, plus a metrics endpoint summarising kitchen throughput.

7.8 Billing & Payment

The billing module computes subtotal, promotional discounts (e.g. WELCOME10, FEAST20), GST (5%), and service charge (5%) to produce a final itemised invoice, and records mock payment completion against each order.

7.9 Cancellation & Rescheduling

Reservations can be cancelled within a defined grace-period policy or rescheduled to a new date/time, automatically releasing or re-checking table-slot availability and logging the cancellation reason for audit purposes.

7.10 Customer Order History

Customers can retrieve their complete order history with itemised receipts, supporting easy review and re-ordering.

7.11 Feedback & Ratings

After a meal, customers submit a 1–5 star rating for food and service quality along with comments; submissions automatically update the corresponding dish's running average rating.

7.12 Multi-Branch Management

Admins and managers maintain multiple restaurant branches, each with its own address, operating hours, seating capacity, tables, and menu, enabling the platform to scale across locations.

7.13 Dashboard & Analytics

Manager and admin users access executive KPI summaries and MongoDB aggregation-pipeline reports covering daily/branch-wise revenue, top-selling dishes, and peak dining hours, visualised on the demo dashboard using Chart.js.

7.14 Responsive Design

The bundled demonstration front end uses a modern, responsive glassmorphism-styled UI so that the customer portal, kitchen display, and manager dashboard remain usable across desktop and mobile viewport sizes.

8. System Architecture

The system follows a layered backend architecture. The Client & Demo Layer consists of the Customer Portal, Kitchen Display, Manager/Admin Dashboard, and a Postman API client, all communicating over HTTP/REST. Requests first pass through the Express Middleware & Security layer — CORS and Helmet security headers, JWT authentication with RBAC filtering, Express-Validator schema checks, and a centralised error handler — before reaching the Core Application Modules.

Thirteen functional modules make up the service layer: authentication and profiles, menu catalogue, table inventory, the reservation slot engine, order placement, the order-workflow state machine, the kitchen FIFO queue, the billing and tax engine, the cancellation policy, customer history, feedback and ratings, multi-branch management, and manager aggregation analytics. Each module reads and writes to a dedicated MongoDB collection in the persistence layer (users, branches, tables, menuItems, reservations, orders, feedback), and any errors raised by a module are caught and formatted consistently by the centralised error handler before returning to the client.

Layer	Components
Client & Demo Layer	Customer Portal, Kitchen Display (KDS), Manager/Admin Dashboard, Postman API Client
Middleware & Security	CORS & Helmet, JWT Auth + RBAC filter, Express-Validator schemas, centralised error handler

Layer	Components
Core Application Modules (13)	Auth & Profiles, Menu Catalog, Table Inventory, Reservation Slot Engine, Order Placement, Order Workflow, Kitchen FIFO Queue, Billing & Tax Engine, Cancellation Policy, Customer History, Feedback & Ratings, Branch Management, Manager Analytics
Database (MongoDB) Layer	users, branches, tables, menuItems, reservations, orders, feedback collections

9. Database Design

The data model spans seven MongoDB collections — users, branches, tables, menuItems, reservations, orders, and feedback — linked through a mix of referencing and embedding, chosen deliberately per entity rather than applied uniformly.

Collection	Key Fields
users	_id, name, email (unique), passwordHash, role (customer kitchen manager admin), phone, branchId (FK)
branches	_id, name (unique), address, city, phone, seatingCapacity, openingTime, closingTime
tables	_id, branchId (FK), tableNumber, capacity, locationZone, isAvailable
menuItems	_id, branchId (FK), name, category, price, isVeg, isAvailable, preparationTime, ratingAverage
reservations	_id, customerId (FK), branchId (FK), tableId (FK), reservationDate, timeSlot, guestsCount, status
orders	_id, orderNumber (unique), customerId (FK), branchId (FK), tableId (FK), orderType, items[] (embedded snapshot), subtotal, discountAmount, taxAmount, serviceCharge, totalAmount, status, statusHistory[] (embedded)
feedback	_id, orderId (FK, unique), customerId (FK), branchId (FK), rating, foodRating, serviceRating, comment

9.1 Embedding vs. Referencing Decisions

- Embedded line items inside orders (items[]): menu-item name, price, and instructions are frozen as an immutable snapshot at order time, so a later menu price change never alters a historical receipt; this also avoids a join against menuItems on every order read.
- Embedded status-history inside orders (statusHistory[]): an order's lifecycle timestamps, status values, and staff IDs are read and updated together with the order document, so keeping them embedded avoids multi-collection writes.
- Referencing for users, branches, and tables: these entities have independent lifecycles, can scale to thousands of records, and are shared across many reservations and orders, so referencing avoids unbounded document growth and data duplication.

10. Security & Validation

- Passwords are hashed with bcrypt.js before storage; plaintext passwords are never persisted.
- Authentication uses signed JSON Web Tokens with a configurable expiry (JWT_EXPIRE), verified on every protected route.

- Role-based access control middleware restricts each endpoint to the appropriate persona(s) — customer, kitchen, manager, or admin.
- Helmet middleware sets protective HTTP security headers, and CORS is configured to control cross-origin access.
- Express-Validator schemas validate and sanitise all incoming request bodies and query parameters before they reach controller logic.
- A centralised error-handling middleware normalises error responses (e.g. HTTP 409 `TABLE_SLOT_CONFLICT` for booking collisions, HTTP 400 `INVALID_STATUS_TRANSITION` for illegal order-status changes) instead of leaking stack traces.
- Sensitive configuration (database URI, JWT secret, tax and service-charge rates) is externalised via environment variables and never committed to source control.

11. Testing

The backend is validated through an automated Jest and Supertest integration test suite covering all thirteen functional modules, run via `npm test` against a seeded or in-memory MongoDB instance. In addition, a Postman Collection (v2.1 format, stored under `postman/restaurant_api.postman_collection.json`) organises requests into nine folders spanning all modules, with pre-request and test scripts that automatically capture JWT tokens and resource IDs for end-to-end manual and automated API verification without manual copying of tokens.

Representative scenarios verified include: rejection of overlapping table reservations with HTTP 409 (`TABLE_SLOT_CONFLICT`); enforcement of the valid order-status transition map so that, for example, a Served order cannot revert to Placed (HTTP 400, `INVALID_STATUS_TRANSITION`); correct GST and service-charge computation in the billing module; and correct aggregation output for the manager sales, popular-dishes, and peak-hours reports.

12. Outcome

The project delivered a fully functional, thirteen-module restaurant backend with a working demonstration front end, five pre-seeded demo accounts spanning all four roles, and a zero-configuration startup path (automatic in-memory MongoDB fallback and an `npm run seed` script) that lets the entire system be evaluated without manual database setup.

The implementation demonstrates practical application of Express.js middleware chaining, Mongoose schema design with justified embedding/referencing trade-offs, JWT-based authentication and RBAC, MongoDB aggregation pipelines for analytics, and automated testing — directly addressing the restaurant double-booking, kitchen miscommunication, and reporting gaps identified in the problem statement.

13. Future Enhancements

- Integrate a real payment gateway (e.g. Razorpay, Stripe) in place of the current mock payment recording.

- Add real-time push updates (WebSockets/Socket.IO) so kitchen and customer order-status changes reflect instantly without polling.
- Send SMS/email notifications for reservation confirmations, cancellations, and order-status milestones.
- Build dedicated native or cross-platform mobile applications for customers and kitchen staff.
- Extend analytics with predictive demand forecasting and inventory/stock management integration.
- Add multi-language support and localisation for menus and the customer-facing UI.

14. References

- GitHub — <https://github.com/aliciaa-20/book-my-bite>
- Node.js Documentation — <https://nodejs.org/docs>
- Express.js Documentation — <https://expressjs.com>
- MongoDB Documentation — <https://www.mongodb.com/docs>
- Mongoose ODM Documentation — <https://mongoosejs.com/docs>
- JSON Web Tokens (JWT) — <https://jwt.io/introduction>
- bcrypt.js — <https://www.npmjs.com/package/bcryptjs>
- Jest Testing Framework — <https://jestjs.io>
- Postman API Platform — <https://www.postman.com>

Screenshots

```

1 public > index.html > ...
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8" />
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>The Royal Bistro – Restaurant Table Reservation & Food Ordering</title>
8   <!-- Modern Google Fonts -->
9   <link rel="preconnect" href="https://fonts.googleapis.com" />
10  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
11  <link href="https://fonts.googleapis.com/css2?family=Cinzel:wght@400;500;600;700&display=block" rel="stylesheet" />
12
13  <!-- Font Awesome Icons & Chart.js -->
14  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/chart.js" />
15  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
16
17  <!-- Custom Stylesheet -->
18  <link rel="stylesheet" href="/css/styles.css" />
19 </head>
20 <body class="dark-theme">
21   <!-- Top Navigation Bar -->
22   <header class="navbar">
23     <div class="nav-container">
24       <div class="brand">
25         <div class="brand-logo-emblem" title="The Royal Bistro">
26           <svg class="royal-crest-svg" viewBox="0 0 44 44" fill="none">
27             <defs>
28               <linearGradient id="goldGrad" x1="0" y1="0" x2="44" y2="44">
29                 <stop offset="0%" stop-color="#FFD700"/>
30                 <stop offset="35%" stop-color="#FFD700"/>
31                 <stop offset="70%" stop-color="#FFD700"/>
32                 <stop offset="100%" stop-color="#FFD700"/>
33               </linearGradient>
34               <linearGradient id="shineGrad" x1="8" y1="18" x2="44" y2="44">
35                 <stop offset="0%" stop-color="white"/>
36                 <stop offset="35%" stop-color="white"/>
37                 <stop offset="70%" stop-color="white"/>
38                 <stop offset="100%" stop-color="white"/>
39               </linearGradient>
40             </defs>
41             <rect width="44" height="44" />
42             <rect width="44" height="44" />
43             <rect width="44" height="44" />
44             <rect width="44" height="44" />
45             <rect width="44" height="44" />
46             <rect width="44" height="44" />
47             <rect width="44" height="44" />
48             <rect width="44" height="44" />
49             <rect width="44" height="44" />
50             <rect width="44" height="44" />
51             <rect width="44" height="44" />
52             <rect width="44" height="44" />
53             <rect width="44" height="44" />
54             <rect width="44" height="44" />
55             <rect width="44" height="44" />
56             <rect width="44" height="44" />
57             <rect width="44" height="44" />
58             <rect width="44" height="44" />
59             <rect width="44" height="44" />
60             <rect width="44" height="44" />
61             <rect width="44" height="44" />
62             <rect width="44" height="44" />
63             <rect width="44" height="44" />
64             <rect width="44" height="44" />
65             <rect width="44" height="44" />
66             <rect width="44" height="44" />
67             <rect width="44" height="44" />
68             <rect width="44" height="44" />
69             <rect width="44" height="44" />
70             <rect width="44" height="44" />
71             <rect width="44" height="44" />
72             <rect width="44" height="44" />
73             <rect width="44" height="44" />
74             <rect width="44" height="44" />
75             <rect width="44" height="44" />
76             <rect width="44" height="44" />
77             <rect width="44" height="44" />
78             <rect width="44" height="44" />
79             <rect width="44" height="44" />
80             <rect width="44" height="44" />
81             <rect width="44" height="44" />
82             <rect width="44" height="44" />
83             <rect width="44" height="44" />
84             <rect width="44" height="44" />
85             <rect width="44" height="44" />
86             <rect width="44" height="44" />
87             <rect width="44" height="44" />
88             <rect width="44" height="44" />
89             <rect width="44" height="44" />
90             <rect width="44" height="44" />
91             <rect width="44" height="44" />
92             <rect width="44" height="44" />
93             <rect width="44" height="44" />
94             <rect width="44" height="44" />
95             <rect width="44" height="44" />
96             <rect width="44" height="44" />
97             <rect width="44" height="44" />
98             <rect width="44" height="44" />
99             <rect width="44" height="44" />
100            <rect width="44" height="44" />
101            <rect width="44" height="44" />
102            <rect width="44" height="44" />
103            <rect width="44" height="44" />
104            <rect width="44" height="44" />
105            <rect width="44" height="44" />
106            <rect width="44" height="44" />
107            <rect width="44" height="44" />
108            <rect width="44" height="44" />
109            <rect width="44" height="44" />
110            <rect width="44" height="44" />
111            <rect width="44" height="44" />
112            <rect width="44" height="44" />
113            <rect width="44" height="44" />
114            <rect width="44" height="44" />
115            <rect width="44" height="44" />
116            <rect width="44" height="44" />
117            <rect width="44" height="44" />
118            <rect width="44" height="44" />
119            <rect width="44" height="44" />
120            <rect width="44" height="44" />
121            <rect width="44" height="44" />
122            <rect width="44" height="44" />
123            <rect width="44" height="44" />
124            <rect width="44" height="44" />
125            <rect width="44" height="44" />
126            <rect width="44" height="44" />
127            <rect width="44" height="44" />
128            <rect width="44" height="44" />
129            <rect width="44" height="44" />
130            <rect width="44" height="44" />
131            <rect width="44" height="44" />
132            <rect width="44" height="44" />
133            <rect width="44" height="44" />
134            <rect width="44" height="44" />
135            <rect width="44" height="44" />
136            <rect width="44" height="44" />
137            <rect width="44" height="44" />
138            <rect width="44" height="44" />
139            <rect width="44" height="44" />
140            <rect width="44" height="44" />
141            <rect width="44" height="44" />
142            <rect width="44" height="44" />
143            <rect width="44" height="44" />
144            <rect width="44" height="44" />
145            <rect width="44" height="44" />
146            <rect width="44" height="44" />
147            <rect width="44" height="44" />
148            <rect width="44" height="44" />
149            <rect width="44" height="44" />
150            <rect width="44" height="44" />
151            <rect width="44" height="44" />
152            <rect width="44" height="44" />
153            <rect width="44" height="44" />
154            <rect width="44" height="44" />
155            <rect width="44" height="44" />
156            <rect width="44" height="44" />
157            <rect width="44" height="44" />
158            <rect width="44" height="44" />
159            <rect width="44" height="44" />
160            <rect width="44" height="44" />
161            <rect width="44" height="44" />
162            <rect width="44" height="44" />
163            <rect width="44" height="44" />
164            <rect width="44" height="44" />
165            <rect width="44" height="44" />
166            <rect width="44" height="44" />
167            <rect width="44" height="44" />
168            <rect width="44" height="44" />
169            <rect width="44" height="44" />
170            <rect width="44" height="44" />
171            <rect width="44" height="44" />
172            <rect width="44" height="44" />
173            <rect width="44" height="44" />
174            <rect width="44" height="44" />
175            <rect width="44" height="44" />
176            <rect width="44" height="44" />
177            <rect width="44" height="44" />
178            <rect width="44" height="44" />
179            <rect width="44" height="44" />
180            <rect width="44" height="44" />
181            <rect width="44" height="44" />
182            <rect width="44" height="44" />
183            <rect width="44" height="44" />
184            <rect width="44" height="44" />
185            <rect width="44" height="44" />
186            &lt
```

```

6 :root {
7   --bg-main: #0b0f19;
8   --bg-surface: #111827;
9   --bg-card: rgba(19, 27, 46, 0.85);
10  --bg-card-hover: rgba(28, 38, 64, 0.95);
11  --border-color: rgba(255, 255, 255, 0.08);
12  --border-focus: #f59e0b;
13
14  --primary: #f59e0b;
15  --primary-hover: #d97706;
16  --primary-glow: rgba(245, 158, 11, 0.25);
17  --secondary: #3b82f6;
18  --secondary-hover: #2563eb;
19
20  --text-main: #f8fafc;
21  --text-muted: #94a3b8;
22  --text-dim: #64748b;
23
24  --success: #10b981;
25  --warning: #f59e0b;
26  --danger: #ef4444;
27  --info: #06b6d4;
28  --purple: #8b5cf6;
29
30  --radius-sm: 6px;
31  --radius-md: 10px;
32  --radius-lg: 16px;
33  --radius-full: 9999px;
34
35  --shadow-sm: 0 2px 8px rgba(0, 0, 0, 0.3);
36  --shadow-md: 0 8px 24px rgba(0, 0, 0, 0.4);
37  --shadow-lg: 0 16px 36px rgba(0, 0, 0, 0.6);
38  --glass-shadow: 0 8px 32px rgba(0, 0, 0, 0.37);

```

```

5
6 require('dotenv').config();
7 const path = require('path');
8 const express = require('express');
9 const cors = require('cors');
10 const helmet = require('helmet');
11 const morgan = require('morgan');
12 const { connectDB } = require('./config/db');
13 const apiRoutes = require('./routes');
14 const errorHandler = require('./middleware/errorHandler');
15
16 const app = express();
17 const PORT = process.env.PORT || 5000;
18
19 const { autoSeedIfEmpty } = require('./utils/seeder');
20
21 // Connect to Database & Auto-Seed mock data if empty
22 if (process.env.NODE_ENV !== 'test') {
23   connectDB().then(() => {
24     autoSeedIfEmpty();
25   });
26 }
27
28 // Security & Utility Middleware
29 app.use(
30   helmet({
31     contentSecurityPolicy: false, // Disabled for local CDN / Chart.js in demo UI
32     crossOriginEmbedderPolicy: false
33   })
34 );
35 app.use(cors());
36 app.use(express.json());
37 app.use(express.urlencoded({ extended: true }));

```



THE ROYAL BISTRO

The Royal Bistro - Indiranagar (Bangalore)

PERSONA:

Customer

Kitchen

Manager

Admin

 0

John Doe

CUSTOMER

Browse Menu & Order

Book Table Slot

Kitchen Display Queue 0

My History & Feedback

Analytics & Management

Dining History & Reviews

Track your active food orders, review previous reservations, view invoices, and share your feedback.

My Food Orders

My Table Bookings

My Reviews & Ratings

Order #ORD-20260904-5807

Served

4/9/2026, 5:38:31 pm • Takeaway

1x Slow Roasted Lamb Shank, 1x Signature Passionfruit Mint Mocktail, 1x Classic Tiramisu, 1x Smoked Peri Peri Chicken Wings, 1x Garlic Butter Naan Basket

Total: ₹1916.25

Invoice

Review

Order #ORD-20260904-1001

Served

4/9/2026, 4:33:52 pm • Dine-In

2x Butter Chicken Grand Cru, 4x Garlic Butter Naan Basket, 2x Signature Passionfruit Mint Mocktail

Total: ₹2103.20

Invoice

Review



THE ROYAL BISTRO

The Royal Bistro - Indiranagar (Bangalore)

PERSONA:

Customer

Kitchen

Manager

Admin

 0

Chef Alessandro (Kitchen Head)

KITCHEN

Kitchen Display System (KDS)

Real-time FIFO queue with preparation timers, cooking notes, and instant lifecycle advancement.

Refresh Queue

LIVE TICKETS

1

New Placed Orders

1

In Cooking / Preparation

0

Ready for Service

1

Served Today

1. Placed (Queue)

1

#ORD-20260904-5807 0m ago (35m left)

T-04 Takeaway John Doe

1x Slow Roasted Lamb Shank

1x Signature Passionfruit Mint Mocktail

1x Classic Tiramisu

1x Smoked Peri Peri Chicken Wings

1x Garlic Butter Naan Basket

Start Cooking

2. In Preparation

1

#ORD-20260904-1002 30m ago (0m left)

VIP-01 Dine-In Sarah Jenkins

2x Crispy Truffle Fries

2x Dum Hyderabad Mutton Biryani

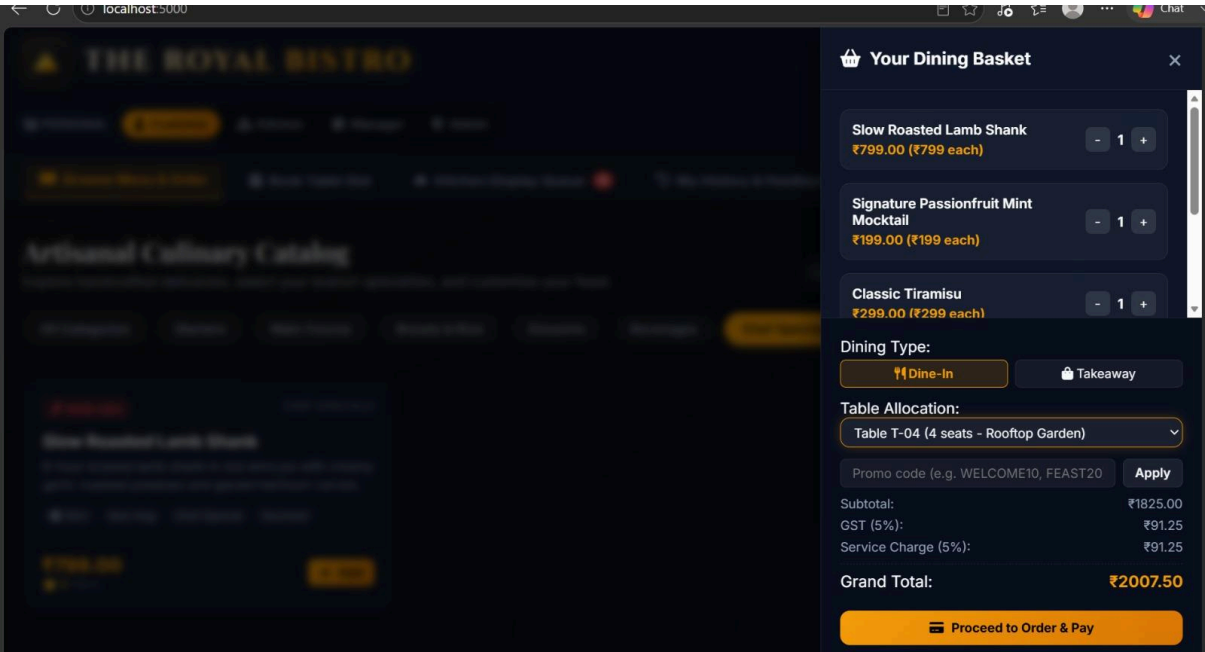
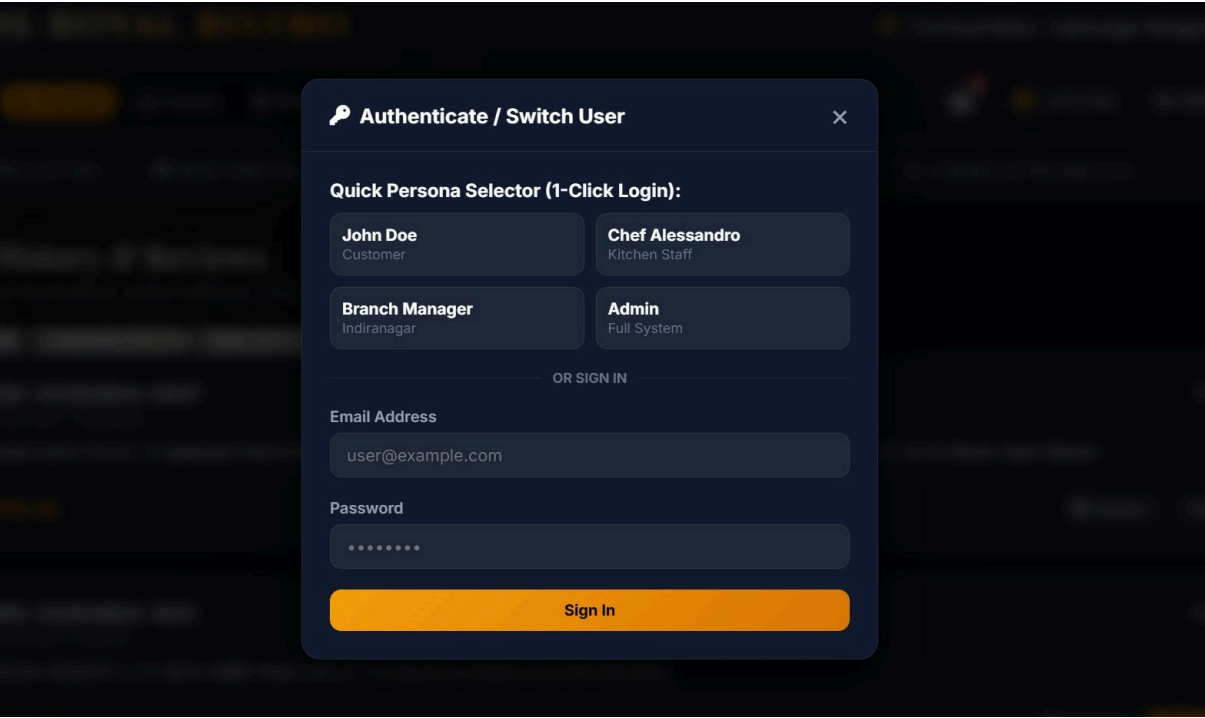
2x Belgian Molten Lava Cake

Mark as Ready

3. Ready to Serve

0

No active orders in this column.





THE ROYAL BISTRO

The Royal Bistro - Indiranagar (Bangalore)

PERSONA:

Customer

Kitchen

Manager

Admin

 0

John Doe

CUSTOMER

Browse Menu & Order

Book Table Slot

Kitchen Display Queue 0

My History & Feedback

Analytics & Management

Reserve Your Dining Table

Conflict-free slot reservation with instant table allocation and zone preferences.



Select Date & Party Size

Restaurant Branch

The Royal Bistro - Indiranagar (Bangalore)

Reservation Date

04-09-2026

Party Size (Guests)

2 Persons (Duo)

Preferred Dining Zone

Any Available Zone

Available Dining Time Slots (90 Mins)

12:00

12:30

13:00

13:30

14:00

14:30

18:00

18:30

19:00

19:30

20:00

20:30

21:00

21:30

Select Specific Table (Optional)



Live Table Layout & Zones

Indoor Main Hall (Climate Controlled)

Outdoor Patio (Al Fresco)

Rooftop Garden (Skyline View)

VIP Lounge (Private Dining)



Booking Summary Preview

Selected Branch:

The Royal Bistro - Indiranagar

Date & Slot:

2026-09-04 at 12:00

Guest Count:

2 Guests

Table Allocation:

Auto Assign



THE ROYAL BISTRO

The Royal Bistro - Indiranagar (Bangalore)

PERSONA:

Customer

Kitchen

Manager

Admin

 0

John Doe

CUSTOMER

Browse Menu & Order

Book Table Slot

Kitchen Display Queue 0

My History & Feedback

Analytics & Management

Artisanal Culinary Catalog

Explore handcrafted delicacies, select your branch specialties, and customize your feast.

Search dishes, ingredients...

Veg Only

All Categories

Starters

Main Course

Breads & Rice

Desserts

Beverages

Chef Specials

VEG

BEVERAGES

Iced Artisan Cold Brew

18-hour slow steeped single-origin Arabica coffee served over ice block with oat milk option.

5m

Vegan

Coffee

₹179.00

★ 4.6 (9+)

+ Add

VEG

BEVERAGES

Signature Passionfruit Mint Mocktail

Fresh passionfruit puree shaken with garden mint, lime juice, and sparkling soda over crushed ice.

6m

Vegan

Refreshing

₹199.00

★ 4.7 (11+)

+ Add

NON-VEG

BREADS & RICE

Dum Hyderabadi Mutton Biryani

Aromatic long-grain basmati rice and tender lamb cuts slow-cooked on dum with saffron, mint, and whole spices.

25m

Non-Veg

Bestseller

Spicy

₹549.00

★ 4.9 (42+)

+ Add



THE ROYAL BISTRO

The Royal Bistro - Indiranagar (Bangalore)

PERSONA:

Customer

Kitchen

Manager

Admin

 0

System Administrator

ADMIN

Browse Menu & Order

Book Table Slot

Kitchen Display Queue 0

My History & Feedback

Analytics & Management

Manager Reports & Analytics

Real-time revenue aggregations, top dishes, peak hours, and inventory management.

+ Add Menu Item

+ Add Table

₹6245.85

Today's Revenue

3

Orders Placed Today

2

Active Reservations

5 / 5 (1 Reviews)

Customer Satisfaction

Daily Revenue Trend



Top Selling Dishes



14